

HW#7: Individual Database Project

From Business Rules to a Working Database Application

Individual assignment | Recommended scope 3 to 5 tables

Objective

This assignment applies the complete database development process, beginning with requirements and business rules and ending with a simple practical use of the implemented database. You may continue from HW#6 UML Design, but you must select only a small and meaningful part of that system containing approximately 3 to 5 related tables.

The goal is not to build a large application. The goal is to demonstrate that you can transform requirements and business rules into a correct, normalized, implemented, and usable database.

1 Project Scope

Choose a small database application, for example:

- clinic appointments or patient services
- hotel reservations or event registration
- online shop or restaurant ordering
- library management or equipment borrowing
- project management or another suitable application

Clearly explain:

- the purpose of the system and its intended users
- the information that the database must manage
- the important activities represented by the database
- the part selected from HW#6, if you continue from that assignment

2 Requirements and Business Rules

Write a short description of the system requirements and provide at least 5 clear business rules. The rules must describe relationships and constraints among the data.

Example business rules:

- A customer can place many orders.
- Each order belongs to exactly one customer.
- An order can contain many products, and a product can appear in many orders.
- The quantity of a product is recorded for each order.

Important: Business rules must come from the requirements, not only from observing sample records. They must remain consistent with the UML model, functional dependencies, keys, normalization, and final schema.

3 Top Down Database Design

Create a UML class diagram for the selected scope. The diagram must show:

- 3 to 5 main classes or association classes
- important attributes and identifiers
- associations and multiplicities
- association attributes, when applicable

Briefly explain how the UML model represents the business rules.

4 Functional Dependencies and Keys

Identify the relevant functional dependencies based on the business rules. For example:

CustomerID -> CustomerName, CustomerPhone

ProductID -> ProductName, UnitPrice

(OrderID, ProductID) -> Quantity

For each initial relation:

1. identify the possible candidate key or keys
2. select one candidate key as the primary key
3. identify the required foreign keys
4. briefly explain why the selected key uniquely identifies a row

Keys should be justified using the business rules.

5 Normalization

Select at least one meaningful relation from your application that contains possible redundancy. Show its normalization process from 1NF to at least 3NF:

1. Present the initial relation.
2. Identify its candidate key and selected primary key.
3. List the relevant functional dependencies.
4. Check whether the relation satisfies 1NF.
5. Identify and remove partial dependencies to obtain 2NF.
6. Identify and remove transitive dependencies to obtain 3NF.
7. Present the final normalized relations with primary and foreign keys.

If the selected relation is already in 2NF or 3NF, clearly explain why. The final schema should contain approximately 3 to 5 tables and should be consistent with the UML design.

Required reflection question: Do the top-down UML design and bottom-up normalization lead to the same or equivalent schema? Explain briefly.

6 Database Implementation

Implement the final schema using MySQL or another relational DBMS approved by the instructor. Your SQL script must include:

- CREATE DATABASE, if applicable, and CREATE TABLE statements
- primary-key and foreign-key constraints
- appropriate data types
- relevant NOT NULL, UNIQUE, CHECK, or other constraints
- meaningful sample data

Minimum sample data:

- at least 5 records in each master table
- at least 10 records in each transaction or association table

7 Practical Database Use

Demonstrate one practical use of the database by selecting **one option only**.

Option A Simple SQL Reporting

Create at least 5 meaningful SQL queries that answer useful questions about the application. Collectively, the queries must include:

- a query joining multiple tables
- a query using GROUP BY and an aggregate function
- a query using HAVING
- a query using a subquery or view
- a query that answers an interesting business question

For every query, provide the question, SQL statement, result, and a short interpretation. Queries that only display all records from one table are not sufficient.

Option B Charts or Simple Dashboard [Extra-Score]

Create at least 5 meaningful charts or a simple one-page dashboard using data from your implemented relational database. You may use Power BI, Tableau, Looker Studio, Excel, Metabase, Streamlit, or another suitable reporting tool.

- at least 3 meaningful business questions
- at least 3 appropriate charts or visualizations
- data retrieved from multiple related tables
- at least one aggregation or calculated measure
- at least one filter, slicer, or interactive control, if supported by the selected tool
- a short interpretation of each chart

For each chart, provide the business question, SQL query or data-preparation method, resulting visualization, and a short interpretation. The dashboard does not need advanced design. A dashboard created directly from an unrelated standalone CSV file is not acceptable; it must use the database designed and implemented in this assignment.

Option C Simple Database Application [Extra-Score]

Develop a small web, desktop, or command-line application connected to your implemented database. It must demonstrate:

- adding a new record
- searching for or viewing records
- updating or deleting a record
- displaying 3nmeaningful reports using joined or aggregated data

A simple interface is sufficient. The quality of the database design and correct database operations are more important than a sophisticated interface. Suitable tools include Python, PHP, JavaScript, Java, Streamlit, or another approved tool. Authentication, deployment, and advanced user-interface design are not required.

8 Validation and Testing

Provide evidence that the database works correctly. Include at least:

- one successful insert or update
- one example showing that a database constraint works
- one result involving data from multiple tables
- one screenshot of the final report, visualization, dashboard, or application
- a short explanation of how the result was checked

For example, demonstrate that the DBMS rejects a duplicate primary key, an invalid foreign key, a duplicate value prohibited by UNIQUE, or a value that violates CHECK.

Deliverables

1 Project Report

Submit one PDF report of approximately 5 to 8 pages containing:

- project title and student information
- scope, requirements, and business rules
- UML class diagram
- functional dependencies and keys
- normalization process from 1NF to 3NF
- final relational schema
- implementation screenshots
- reporting results, charts or dashboard, or application screenshots
- testing evidence and a short conclusion

2 SQL File

Submit one runnable .sql file containing table creation, constraints, sample data, and reporting queries when applicable. The script should run from beginning to end without manual correction.

3 Practical Database Use Files

Submit the files corresponding to your selected option: SQL queries and outputs for Option A; chart or dashboard files and data-preparation details for Option B; or application source code and brief running instructions for Option C.

4 Short Demonstration Video

Submit a 5 to 8 minute video demonstrating:

- requirements and business rules
- UML design and normalization
- implemented tables and constraints
- the SQL reports, charts or dashboard, or database application
- one database constraint being tested

Your face should be visible at the beginning of the video, and you must explain the work in your own words !!!!!.

Suggested Folder Structure

```
StudentID_HW7_ProjectName/  
|-- 01_Report/StudentID_HW7_Report.pdf  
|-- 02_Database/StudentID_HW7.sql  
|-- 03_Practical_Database_Use/  
|-- 04_Screenshots/  
`-- README.txt
```

The README file should identify the selected option, DBMS and tools used, and explain how to run or open the submitted SQL reports, dashboard, or application.

Important Notes

- This is an individual assignment.
- Keep the scope small, with approximately 3 to 5 tables.
- You may reuse and improve a selected part of HW#6. You do not need to implement the complete HW#6 system.
- The UML diagram, functional dependencies, normalization, SQL schema, and application must be consistent.
- Business rules must come from the requirements, not only from sample data.
- The application interface does not need to be sophisticated.
- You must be able to explain every table, relationship, constraint, query, and program component submitted.
- **A complex application with an incorrect database design will not receive more credit than a simple application with a correct and well-explained design.**